UNIVERDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
LENGUAJES FORMALES Y DE PROGRAMACION
PROYECTO 2
SEGUNDO SEMESTRE 2026
Fernan Andoni González Hernández 202331283

MANUAL DE TECNICO

INSTITUCION: Ingeniería en ciencias y sistemas cunoc

CURSO: Lenguajes Formales y De Programacion

FECHA: 28/10/2026

REQUISITOS DEL ENTORNO

JAVA JDK 21

NetBeans IDE con proyecto Java Swing.

JFlex para generar la calse lexer.java a partir de la especificacion .flex

1,ARQUITECTURA GENERAL

El procesamineto ser realiza en fases consecutivas. Cada fase consume lo que es la salida de alanterior y acumula sus propios errores en una lista independiente, de modo que el usaurio recibe todos los problemas en una sola ejecucion.

La cual la arquitectura tiene la siguiente idea:



ORGANIZACION DE LOS PAQUETES

PAQUETE	ANALIZADOR	RESPONSABILIDAD
ANALIZADOR	Lexer	El analizadore generado por Jflex. Produce la lista de tokens y errores lexicos.
ANALIZADOR	Parser	El analizador sisntactico descendente y recursivo. Construye el Programay la tabla de simbolos.

ANALIZADOR	AnalizadorSemantico	Recorre el AST y verifica las reglas semanticas
ANALIZADOR	GeneradorPromptZal	Convierte el AST validado en el texto del prompt.
ANALIZADOR	Imprimir AST	Imprime el AST en consola(depuracion).
MODELO	Token, TipoToken	Representacion de un token y enumeracion de sus categorias.
MODELO	Programa, Agente, Comando, Conector, Variable, Directiva, Expresion, Termino.	SON NODOS DEL AST.
MODELO	Simbolo, TablaSimbolo	Tabla de simbolos con ambito.
MODELO	ErrorLexico, ErrorSintactico, error semantico.	Registro de los errores de cada fase.
MODELO	NumeroLineas	Componente Swing que dibuja los numeros de linea del editor
INTERFAZ	IDEPrincipal	Conetiene las componentes de JavaSwing que es la representcaion grafica de nuestro programa

ANALIZADORE LEXICO

El analizador lexico se define en un archivo de especificacion JFlex. La clase generada se llama Lexer, es publica usa lo que es unicode, lleva conteo de linea y columna y devuelve lo que es

objetos token al llegar al final del archivo retorna un token EOF mediante lo que es %eofval.

Funcionamiento interno.

1. **Crear token(tipo token):** se construye un token con numero correlativo, lexema(yytext()), tipo, fila, y columna(ambas desde 1), lo cual guarda en una lista interna y lo retorna.

2. **CrearTokenEOF():** igual que el anterior con lexema EOF.

3. **getTokens():** y **getErrores:** exponene lo que son las listas acumuladas para la interfaz y para el parser.

4. Las reglas de espacios y comentarios no retornan lo que es un token, por lo cual se descartan. Las reglas de error agregan lo que es un ErrorLexico y no retornan, asi el analisis continua con el siguiente caracter.

DEFINICIONES REGULARES

Nombre	Expresion	Descripcion
DIGITO	[0 - 9]	UN DIGITO
LETRA	[a- za-Z_]	LETRA O GUION BAJO
ID	{LETRA} ({LETRA} | {DIGITO})*	IDENTIFICADOR
NUMERO	{DIGITO}+ (\. {DIGITO}+)?	ENTERO O DECIMAL.
CADENA	\" ([^\" \\\r\n] |\\[^\r\n])*\"	CADENA ENTRE COMILLAS, CON ESCAPES, EN UNA SOLA

		LINEA.
CADENA_SIN_CERRAR	\" ([^\" \\\r\n] |\\[^\r\n])*	CADENAS DE APERTURA SIN CIERRE EN LA LINEA.
ESPACIOS	[\t \r \n]+	ESPACIO EN BLANCO
COMENTARIO_LINEA	"//" [^ \r \n]*	COMENTARIO HASTA FINA DE LA LINEA
COMENTRAIO BLOQUE	"/*" ([^*] | * +[^*/]) * \ * + "/"	COMENTARIO DE BLOQUE

TABLA DE TOKENS

TOKEN(TIPO TOKEN)	LEXEMA	CATEGORIA
DIRECTIVA	"@" (modelo, rol, formato)	directiva
AGENTE	AGENTE	Palabra Reservada
VARIABLE	variable	Palabra Reservada
EJECUTAR	EJECUTAR	Palabra Reservada
EXPORTAR	EXPORTAR	Palabra Reservada
COMANDO_IA	PREGUNTAR | GENERAR | RESUMIR | ANALIZAR | TRADUCIR | CLASIFICAR | EXTRAER	Comando de IA
CONECTOR	SOBRE | DESDE | EN | COMO	Conector
CARGAR	CARGAR	Funcion del sistema
CADENA	{CADENA}	LITERAL
NUMERO	(NUMERO)	LITERAL
ID	{ID}	identificador
IGUAL	=	OPERADOR

FLECHA	->	OPERADOR
MAS	+	OPERADOR
LLAVE_A/LLAVE_C	{ }	DELIMITADOR
PAR_A/PAR_C	( )	DELIMITADOR
COMA	,	DELIMITADOR
EOF	Fin de archivo	especial

RESOLUCION DE AMBIGÜEDADES

JFlex aplica lo que es la rega del lexema mas largo y ante empate la regla escrita primero. Pro eso las palabras reservadas, los comandos y los conectores se declaran antes de al regla {ID}: por lo que la palabra AGENTE coincide con ambas reglas con la misma longitud y gana la reservada. Del mismo modo, una cadena correctamente cerrada coincide con CADENA y con CADENA_SIN_CERRAR, pero CADENA es mas larga(incluye la comilla al final) es por eso que prevalece.

Las palabras reservadas distinguen mayusculas y minusculas: contexto y variable van en minusculas; el resto, en mayusculas.

ERRORES LEXICOS

ERROR	CAUSA	EJEMPLO
Cadena sin cerrar	Comilla de apertura sin cierre antes del fin de linea	Contexto = “hola
Caracter no reconocido	Cualquier simbolo que no pertenece al alfabeto del lenguaje.	$ # % -

Lo cual cada error se almacena como ErrorLexico(lexema, tipoError, fila, columna).

ANALISIS SINTACTICO

El parser es de tipo descenso rescursivo predictivo: existe un metodo por cada no terminal de la gramatica y la decision de que produccion aplicar se toma observando unicamente el token actual(tokenActual) recibe la lista de tokens producidas por el lexer.

GRAMATICA

la gramatica que se presnta. Los terminales son los nombres de tipoToken. “ϵ” indica una produccion que no consume tokens

<programa::= <directivas> <agentes> <ejecuciones> <exportar> EOF

<directivas> ::= DIRECTIVA CADENA <directivas> | ϵ

<agentes> ::= <agente> <masAgentes>

<masAgentes> ::= <agente> <masAgentes> | ϵ

<agente>::= AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C

<contexto> ::= CONTEXTO IGUAL CADENA

<variables> ::= VARIABLE ID IGUAL <expresion> <variables> | ϵ

<comandos> ::= <comando> <masComandos>

<masComandos> ::= <comando> <masComandos> | ϵ

<comando> ::= COMANDO_IA <expresion> <conectores> FLECHA ID

<conectores> ::= CONECTOR <expresion> <conectores> | ϵ

<expresion> ::= <termino> <masTerminos>

<masTerminos> ::= MAS <termino> <masTerminos> | ϵ

<termino> ::= CADENA | ID | NUMERO <unidad> | <cargar>

<unidad> ::= ID | ϵ

<cargar> ::= CARGAR PAR_A CADENA PAR_C

<ejecuciones> ::= EJECUTAR ID <masEjecuciones>

<masEjecuciones>::= EJECUTAR ID <masEjecuciones> | ϵ

<exportar> ::= EXPORTAR ID <masIds>

<masIds> ::= COMA ID <masIds> | ϵ

METODOS PARSER

METODO	FUNCION
Avanzar()	Mueve el token acutal al siguiente token; se detiene en el utlimo(EOF).
Consumir(tipo)	Si el token actual coincide, avanza y retorna true; si no, registra un ErrorSintactico y retorna false,
registrarError()	Crea un errorSintactico con lexema encontradp, lo esperado, fila y columna
programa()	Simbolo inciial. Exige las secciones de ejecucion y exportacion y verificaEOF
Directivas()	Lee pares DIRECTIVAS CADENAS y los agrega al programa(sin comillas)
Agentes() / masAgentes() / agente()	Procesan uno o mas agentes; crea agnete y registran el simbolo de tipo AGENTE
Contexto(agente)	Procesa contexto = ".." retorna false si falla para activar lo que es la recuperacion
Variables(agente)	Procesa declaraciones varibale id = expresion; registra simbolos de tipo VARIABLE.
Comando()/comandos()	Procesan comandos de IA con sus conectores y el destino; registrarn RESULTADO_COMANDO
Conectores(comando)	Procesa cero o mas pares CONECTORES expresion.
Expresion()/ masTerminos() / termino()	Procesan expresiones con concatenacion (+) y constituyen los terminos.
Unidad()	Consume ID opcional que sigue a un

	NUMERO
Cargar()	Procesa CARGAR(“archivo”) retorna el nombre del archivo o null si hay error.
Ejecuciones() / masEjecuciones()	Procesan una o mas sentencias EJECUTAR id.
Exportar()	Procesan EXPORTAR id(id)*

RECUPERACION DE ERRORES (modo panico)

Cuando se detecta un error, el parser lo registra y descarta tokens hasta encontrar uno de sincronizacion, para continuar el analisis y reporetar errores con la misma corrida. Cada contexto usa un conjunto distinto.

METODO	TOKENS DE SINCRONIZACION	USADO CUANDO
Recuperar()	AGENTE, LLAVE_C, COMANDO_IA, VARIABLE, EJECUTAR, EXPORTAR, EOF	Error dentro de varibales o terminos
recuperarContexto()	VARIABLE, COMANDO_IA, LLAVE_C, AGENTE, EJECUTAR, EXPORTAR, EOF	Contexto mal formado o ausente
recuperarComando	COMANDO_IA, LLAVE_C, EJECUTAR, EXPORTAR, EOF	Comando incompleto o invalido
RecuperarNivelSuperior()	EJECUTAR, EXPORTAR, EOF	Faltan secciones de nivel
Bucles en agente()/agentes()	AGENTE, EJECUTAR, EXPORTAR, EOF	Agente sin nombre, sin llave de apertura o sin cierre,

VARIABLE DE SINTAXIS ABSTRACTA(AST)

El parser construye el AST con las clases del paquete de modelos. La cual la raiz es Programa.

CLASE	ATRIBUTOS PRINCIPALES	DESCRIPCION
Programa	Directivas, agentes, ejecuciones, exportaciones	Raiz del arbol, guarda la fila de cada ejecucion y exportacion para reportar errores.
directiva	Nombre, valor, fila	@modelo “GPT 4”
agente	Nombre, contexto, fila, variable, comandos	Unidad principal del lenguaje
varibale	Nombre, expresion, fila	Declaracion varibale id = expresion
comando	Tipo, expresion, conectores, destino, fila, filaDestino	Instruccion de IA que guarda el resultado en destino
conector	Tipo, expresion	Par SOBRE/DEDE/EN/COMO con su expresion
expresion	Items(lista de termino) valida	Concatenacion de terminos con operador +
termino	Tipo, valor, fila	Tipos: CADENA, VARIABLE, NUMERO, CARGAR, TIPO_CONTENIDO.

Metodos relevantes de Expresion obtenerTexto() representacion textual usada por IMPRIMIRAST.

TABLA SIMBOLOS

Durante el parseo se registra un simbolo(nombre, tipo, ambito, fila) por cada declaracion:

Tipo de Simbolo	Se registra cuando	ambito
AGENTE	Se declara un agente	GLOBAL
VARIABLE	Se declara varibale id = ...	Nombre de agente
RESULTADO_COMANDO	Un comando gurada su resultado con ->id	Nombre del agente

TablaSimbolo ofrece lo qeu es agregarSimbolo, getSimbolo, existeAgente, existeVariable, existeResultado y eliminarSimbolosAgente. El analizadore semantico utiliza para validar las exportaciones.

ANALISIS SEMANTICO

Analizadore semantico recibe el programa y la tablaSimbolos, ejecuta siete verificaciones en orden y acumula los errores como errorSemantico(descripcion, fila) no se detiene en el primer error.

No	Metodo	Regla verificada	mensaje
1	verificarContextos	El contexto no puede estar vacio.	El agente X debe tener un contexto con contenido
2	verificarAgentesDuplicados	No pueden existir dos agnetes con el mismo nombre	El agente x esta duplicado
3	verificarVariablesDuplicadas	Dentro de un agente no se repite el nombre	La variable x ya fue declarada en el agente A
4	verificarResultados	El destino de un coando	El resultado r ya

	Duplicados	no puede repetirese ni coincidir con una variable del mismo agente	existe como variabled
5	VerificarEjecuciones	Todo EJECUTAR debe referires a un agente existente	Se intenta ejecutar el agente x pero no existe
6	verificarVariables	Toda variable usada debe haberse declarado antes	La variable x no ha sido decladrada en el agente A
7	verificarExportaciones	Lo exportado no debe repetirse y debe ser variable o resultado de un agente ejecutado	No se puede exportar x por que no pertenece a

GENERACION DEL PORMP

generadorPromptZal.generar(Programa) produce el texto final. Solo se incluye los agentes que aparecen en EJECUTAR.

Encabezado # y si existen las directivas @modelo @formato @rol

Una seccion ## agente: nombre agnete aejecutado, con contexto, variables disponibles e instrucciones enumeradas.

Una seccion final ## Resultado Solicitado con la lista de Exportaciones.

TRADUCCION DE COMANDOS

COMANDO	CONECTORES USADOS	INSTRUCCION GENERADA
PREGUNTAR	**SOBRE, DESDE**	Basandose en <sobre>, utilizando la informacion de <desde> responde <pregunta>
GENERAR	**DESDE**	Genera <contenido> a partir de <desde>
RESUMIR	**EN**	Resume <texto> en un maximo de <en>
ANALIZAR	**SOBRE**	Analiza <texto> considerado

		<sobre>
TRADUCIR	DESDE, EN	Traduce <texto> utilizando el contendio de <desde> al idioma <en>
CLASIFICAR	EN	Clasifica <texto> en categorias <en>
EXTRAER	DESDE	Extrae <texto> a partir de <desde>

Todas las instrucciones terminan con guardar el resultado como <destino> Si el comando tiene el conector COMO se añade entrega el resultado en formato<como>. Los teminos cargar se redatcan como el archivo 'nombre'

Ejemplo

```
@modelo "GPT-4"
@rol "Analista"
@formato "Markdown"

AGENTE Resumidor {
    contexto = "Eres un experto en resumir"
    variable doc = CARGAR("informe.txt")
    RESUMIR doc EN 100 palabras -> resumen
}

EJECUTAR Resumidor
EXPORTAR resumen
```

```
# Prompt generado con PromptZal
Modelo destino: GPT-4
Rol general: Analista
Formato de respuesta: Markdown

## Agente: Resumidor
Contexto: Eres un experto en resumir
Variables disponibles:
- doc: contenido del archivo "informe.txt" (adjuntar el archivo)
Instrucciones:
1. Resume doc en un máximo de 100 palabras. Guarda el resultado como resumen.

## Resultado solicitado
- resumen
```